databricks

Building Lakeflow Spark Declarative Pipelines

LECTURE

Change Data Capture (CDC) Overview

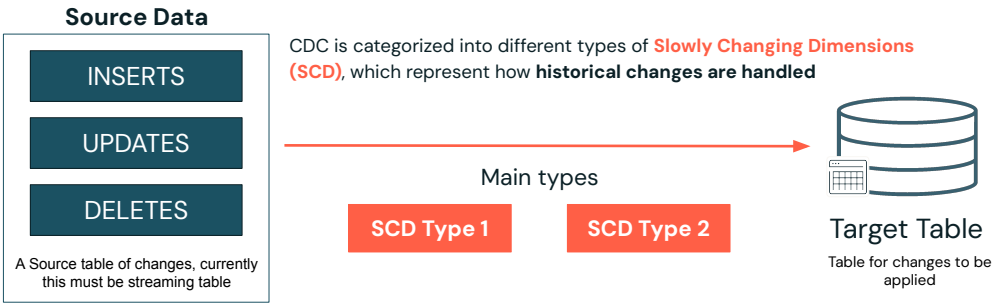
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

In this lecture, you will learn the fundamentals of Change Data Capture (CDC), including what it is and how to implement Slowly Changing Dimensions (SCD) Type 1 and Type 2 to manage and track evolving data in your pipelines.

Change Data Capture (CDC) Overview

What is Change Data Capture?

- Change Data Capture (CDC) is a technique **used to track and capture changes in a data source** (such as a database, lakehouse or data warehouse).
- Then **applying those changes** into the Lakehouse or data warehouse



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's talk about Change Data Capture, or CDC, a foundational concept for keeping data in sync across systems.

So, what exactly is CDC?

Change Data Capture (CDC) is a technique used to track and capture changes in a data source (such as a database, lakehouse or data warehouse). Those changes are then applied to a target table, for example, your Lakehouse, to keep it up to date with the latest state from the source.

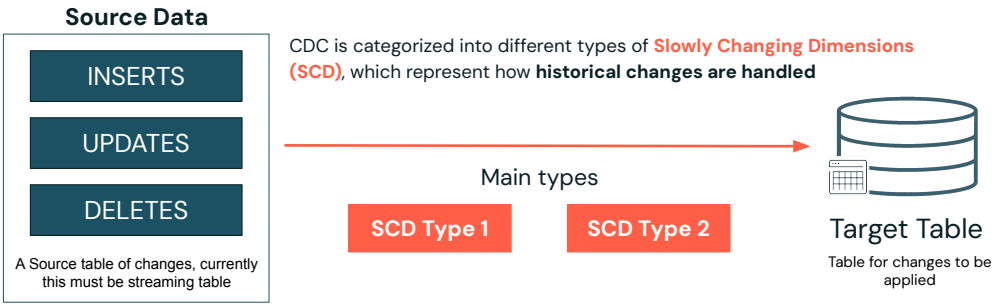
CDC is also closely related to how we handle Slowly Changing Dimensions, or SCDs, which define how historical changes are tracked and stored in your target. There are two main types of SCD we'll focus on:

- **SCD Type 1** – Overwrites existing data (no history tracking)
- **SCD Type 2** – Tracks historical changes by storing previous versions of records

Change Data Capture (CDC) Overview

What is Change Data Capture?

- Change Data Capture (CDC) is a technique **used to track and capture changes in a data source** (such as a database, lakehouse or data warehouse).
- Then **applying those changes** into the Lakehouse or data warehouse



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's walk through a high-level example to make this concrete.

Imagine we're working with a **customer** table.

- Our source data contains new customer records, as well as updates and deletes to existing customers.
- We want to apply those changes to our target table using either SCD Type 1 or SCD Type 2 logic (We'll dive deeper into what those types mean and how they're implemented shortly) to keep our target customers table up to date with the latest information.

Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 1 (1 of 2)

SCD Type 1 (Overwrite Target with Latest Values)

- When a record updates, the **previous record is simply overwritten** by its **key** with the new value.
- When a record is deleted by its **key**, the **record is removed**
- There is **no tracking of old keys (rows),** only the **current data** is retained.

customers (*target table with all customers*)

CustomerID	Name	Address	ProcessDate
1	Peter	1 Blue Rd.	5/1/2025
2	Samarth	22 Front St	5/1/2025

*Update the **customers** table with the **new customer** information*

updates (*source table with updates*)

CustomerID	Name	Address	Operation	ProcessDate
1	Peter	**99 Sunny St..**	**UPDATE**	5/15/2025
1	Peter	**123 Main St.**	**UPDATE**	5/20/2025
2	null	null	**DELETE**	5/20/2025
3	Kostas	100 Athens Dr.	**NEW**	5/20/2025

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's start with an overview of Slowly Changing Dimension Type 1, or SCD Type 1.

SCD Type 1 Overview (Target table is overwritten with the latest values)

- When a record is updated by it's key(s), the existing row is overwritten in the target table and no version history is kept.
- When a record is deleted by it's key(s), it is removed from the target table and ho version history is kept.
- Only the current state of each record is retained in the target table, no tracking of previous values or changes.

Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 1 (1 of 2)

SCD Type 1 (Overwrite Target with Latest Values)

- When a record updates, the **previous record is simply overwritten** by its **key** with the new value.
- When a record is deleted by its **key**, the **record is removed**
- There is **no tracking of old keys (rows),** only the **current data** is retained.

customers (*target table with all customers*)

CustomerID	Name	Address	ProcessDate
1	Peter	1 Blue Rd.	5/1/2025
2	Samarth	22 Front St	5/1/2025

*Update the **customers** table with the **new customer** information*

updates (*source table with updates*)

CustomerID	Name	Address	Operation	ProcessDate
1	Peter	**99 Sunny St..**	**UPDATE**	5/15/2025
1	Peter	**123 Main St.**	**UPDATE**	5/20/2025
2	null	null	**DELETE**	5/20/2025
3	Kostas	100 Athens Dr.	**NEW**	5/20/2025

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's apply this to our scenario:

- We have a **customers** table as our target. The table currently contains two customers:
  - customer_id 1, Peter
  - customer_id 2, Samarth
- We have an **updates** table as our source, this contains:
  - Updates (Peter has had two update on his address. One on 5/15 and the other on 5/20, customer_id 1)
  - Deletes (Samarth wants to be removed, customer_id 2)
  - Inserts (New customer Kostas, customer_id 3)
- Our goal is to update the **customers** table with the new customer information from the **updates** source table.

Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 1 (2 of 2)

customers (*target table with all customers*)

Update the row

Delete the row

Insert the row

CustomerID	Name	Address	ProcessDate
1	Peter	1 Blue Rd.	5/1/2025
2	Samarth	22 Front St	5/1/2025
3	*Kostas*	*100 Athens Dr.*	*5/20/2025*

updates (*source table with updates*)

CustomerID	Name	Address	Operation	ProcessDate
1	Peter	99 Sunny St..	UPDATE	5/15/2025
1	Peter	123 Main St.	UPDATE	5/20/2025
2	null	null	DELETE	5/20/2025
3	Kostas	100 Athens Dr.	NEW	5/20/2025

Updated SCD Type 1 Table

CustomerID	Name	Address	ProcessDate
1	Peter	123 Main St.	5/20/2025
3	Kostas	100 Athens Dr.	5/20/2025

- **Peter** was updated with the **latest** information
- **Samarth** was deleted from the table
- **Kostas** was added to the table

Used **SCD Type 1** when only the **current data matters** and historical values are irrelevant

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

When we apply SCD Type 1, our target table is updated with the latest customer information, without keeping any historical versions of the data based on the **CustomerID** (ID column) and **ProcessDate** (sequence column).

Let's walk through what happens in this example:

- Peter (customer_id 1): His address is updated to the latest address based on the ProcessDate to 123 Main St. from the updates table.
- Samarth (customer_id 2): He's been deleted, so his row is removed from the target table.
- Kostas (customer_id 3): He's a new customer, so his record is inserted into the table.

The end result? The **customers** table contains a current snapshot of all active customers, with no history, just the most recent customers.

This is the simplest CDC strategy, and it's ideal when maintaining historical changes isn't necessary. You just need the latest, most accurate data.

Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 2 (*Outside the scope of the course*)

SCD Type 2 (Historical Tracking/Versioning)

- *When a record changes:*
  - The **old record is preserved** with an additional column indicating its validity period (start date, end date, or a current flag) and a **new record is inserted** with the updated information.
  - When a **record is marked as deleted,** the record is kept and a column indicates the record is **inactive**.
- Used when **historical data is important**, and the system needs to track **how attributes change over time** (like tracking changes in a customer's address or status).

Columns indicate **active** and **inactive** rows

Example customers target table with SCD Type 2

CustomerID	Name	Address	ProcessDate	__START_AT	__END_AT
1	Peter	123 Main St.	5/20/2025	5/20/2025	null
1	Peter	1 Blue Rd.	5/1/2025	5/1/2025	5/20/2025
2	Samarth	22 Front St	5/1/2025	5/1/2025	5/20/2025
3	Kostas	100 Athens Dr.	5/20/2025	5/20/2025	null

Null values indicate the **current row**

Non null values indicate **inactive (historic)** rows

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Now let's talk about Slowly Changing Dimensions Type 2, or SCD Type 2, which introduces historical tracking and versioning of records.

SCD Type 2 Overview

- When a record changes
  - The old record is preserved with an additional column indicating its validity period (start date, end date, or a current flag) and a new record is inserted with the updated information.
  - When a record is marked as deleted, the record is kept and a column indicates the record is inactive.
- SCD Type 2 is used when historical data is important, and the system needs to track how attributes change over time (like tracking changes in a customer's address or status).

Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 2 (*Outside the scope of the course*)

SCD Type 2 (Historical Tracking/Versioning)

- *When a record changes:*
  - The **old record is preserved** with an additional column indicating its validity period (start date, end date, or a current flag) and a **new record is inserted** with the updated information.
  - When a **record is marked as deleted,** the record is kept and a column indicates the record is **inactive**.
- Used when **historical data is important**, and the system needs to track **how attributes change over time** (like tracking changes in a customer's address or status).

Columns indicate **active** and **inactive** rows

Example customers target table with SCD Type 2

CustomerID	Name	Address	ProcessDate	__START_AT	__END_AT
1	Peter	123 Main St.	5/20/2025	5/20/2025	null
1	Peter	1 Blue Rd.	5/1/2025	5/1/2025	5/20/2025
2	Samarth	22 Front St	5/1/2025	5/1/2025	5/20/2025
3	Kostas	100 Athens Dr.	5/20/2025	5/20/2025	null

Null values indicate the **current row**

Non null values indicate **inactive (historic)** rows

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's walk through an example of SCD Type 2 in action to see how historical tracking works in practice. In the example table above, we've added two important metadata columns to the target table:

- **__START_AT** – shows when the row became active
- **__END_AT** – shows when the row became inactive (if it has)
  - A null **__END_AT** means the row is currently active.

Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 2 (*Outside the scope of the course*)

SCD Type 2 (Historical Tracking/Versioning)

- *When a record changes:*
  - The **old record is preserved** with an additional column indicating its validity period (start date, end date, or a current flag) and a **new record is inserted** with the updated information.
  - When a **record is marked as deleted,** the record is kept and a column indicates the record is **inactive**.
- Used when **historical data is important**, and the system needs to track **how attributes change over time** (like tracking changes in a customer's address or status).

Columns indicate **active** and **inactive** rows

Example customers target table with SCD Type 2

CustomerID	Name	Address	ProcessDate	__START_AT	__END_AT
1	Peter	123 Main St.	5/20/2025	5/20/2025	null
1	Peter	1 Blue Rd.	5/1/2025	5/1/2025	5/20/2025
2	Samarth	22 Front St	5/1/2025	5/1/2025	5/20/2025
3	Kostas	100 Athens Dr.	5/20/2025	5/20/2025	null

Null values indicate the **current row**

Non null values indicate **inactive (historic)** rows

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's breakdown the example above:

Customer ID 1 – Peter

- Two records exist for Peter:
  - The active record shows Peter's updated address.
    - **__END_AT** is **null** → still active
    - **__START_AT** marks when the update occurred
  - The inactive record holds Peter's old address. We can tell it is inactive because **__END_AT** is populated → no longer current

Customer ID 2 – Samarth

- Since Samarth deleted his account, his existing record is now inactive.
- A date is added to **__END_AT** to indicate when he was removed.

Change Data Capture (CDC) Overview

Slowing Changing Dimensions (SCD) Type 2 (*Outside the scope of the course*)

SCD Type 2 (Historical Tracking/Versioning)

- *When a record changes:*
  - The **old record is preserved** with an additional column indicating its validity period (start date, end date, or a current flag) and a **new record is inserted** with the updated information.
  - When a **record is marked as deleted,** the record is kept and a column indicates the record is **inactive**.
- Used when **historical data is important**, and the system needs to track **how attributes change over time** (like tracking changes in a customer's address or status).

Columns indicate **active** and **inactive** rows

Example customers target table with SCD Type 2

CustomerID	Name	Address	ProcessDate	__START_AT	__END_AT
1	Peter	123 Main St.	5/20/2025	5/20/2025	null
1	Peter	1 Blue Rd.	5/1/2025	5/1/2025	5/20/2025
2	Samarth	22 Front St	5/1/2025	5/1/2025	5/20/2025
3	Kostas	100 Athens Dr.	5/20/2025	5/20/2025	null

Null values indicate the **current row**

Non null values indicate **inactive (historic)** rows

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Customer ID 3 – Kostas

- A new customer, so his record was inserted into the table.
- **__START_AT** shows when he joined
- **__END_AT** is null → the record is currently active

Change Data Capture (CDC) Overview

Using AUTO CDC INTO in Spark Declarative Pipelines (Formerly APPLY CHANGES INTO)

customers (*target table with all customers*)

CustomerID	Name	Address	ProcessDate
1	Peter	1 Blue Rd.	5/1/2025
2	Samarth	22 Front St	5/1/2025

updates (source table with updates)

CustomerID	Name	Address	Operation	ProcessDate
1	Peter	99 Sunny St..	**UPDATE**	5/15/2025
1	Peter	123 Main St.	**UPDATE**	5/20/2025
2	Samarth	22 Front St	**DELETE**	5/20/2025
3	Kostas	100 Athens Dr.	**NEW**	5/20/2025

```
CREATE OR REFRESH STREAMING TABLE customers;

CREATE FLOW scd_type_1_flow AS
AUTO CDC INTO customers
 FROM STREAM updates
 KEYS (CustomerID)
 APPLY AS DELETE WHEN operation = "DELETE"
 SEQUENCE BY ProcessDate
 COLUMNS * EXCEPT (operation)
 STORED AS SCD TYPE 1;
```

- Create a streaming table, then use AUTO CDC to populate it
- Apply updates, inserts and deletes to the **target** table
- Source records to determine **updates, deletes and inserts**
- The column(s) that **uniquely identify a row** in the source and target tables
- Specifies when a CDC event should be treated as a **DELETE** rather than an upsert.
- Specifies the **logical order of CDC events** in the source data
- Specifies a **subset of columns** to include in the **target**
- Whether to store records as **SCD type 1 (default) or type 2**

*INSERTs and UPDATEs are implicitly implemented using the **KEY(s)**, no coding required*

Use **`AUTO CDC INTO (Formerly APPLY CHANGE INTO)`** instead of a traditional batch job with complex **`MERGE INTO`** `syntax`

Use **`AUTO CDC FROM SNAPSHOT`** (Public Preview as of 2025Q2) to process changes in database snapshots.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Now that we've learned about Change Data Capture and both SCD Type 1 and Type 2, let's see how we can easily implement these patterns using Lakeflow Spark Declarative Pipelines (Specifically SCD Type 1).

Lakeflow provides a built-in declarative CDC mechanism using the AUTO CDC INTO statement.

You use AUTO CDC INTO to handle updates, inserts, and deletes in your pipeline. Here's what each part of the statement does:

- AUTO CDC INTO customers – Applies all CDC events to the target table named customers.
- FROM STREAM updates – Specifies the source stream that contains CDC events (inserts, updates, deletes).

Change Data Capture (CDC) Overview

Using AUTO CDC INTO in Spark Declarative Pipelines (Formerly APPLY CHANGES INTO)

customers (*target table with all customers*)

CustomerID	Name	Address	ProcessDate
1	Peter	1 Blue Rd.	5/1/2025
2	Samarth	22 Front St	5/1/2025

updates (source table with updates)

CustomerID	Name	Address	Operation	ProcessDate
1	Peter	99 Sunny St..	UPDATE	5/15/2025
1	Peter	123 Main St.	UPDATE	5/20/2025
2	Samarth	22 Front St	DELETE	5/20/2025
3	Kostas	100 Athens Dr.	NEW	5/20/2025

```
CREATE OR REFRESH STREAMING TABLE customers;

CREATE FLOW scd_type_1_flow AS
AUTO CDC INTO customers
 FROM STREAM updates
 KEYS (CustomerID)
 APPLY AS DELETE WHEN operation = "DELETE"
 SEQUENCE BY ProcessDate
 COLUMNS * EXCEPT (operation)
 STORED AS SCD TYPE 1;
```

- Create a streaming table, then use AUTO CDC to populate it
- Apply updates, inserts and deletes to the **target** table
- Source records to determine **updates, deletes and inserts**
- The column(s) that **uniquely identify a row** in the source and target tables
- Specifies when a CDC event should be treated as a **DELETE** rather than an upsert.
- Specifies the **logical order of CDC events** in the source data
- Specifies a **subset of columns** to include in the **target**
- Whether to store records as **SCD type 1 (default) or type 2**

*INSERTs and UPDATEs are implicitly implemented using the **KEY(s)**, no coding required*

Use **`AUTO CDC INTO (Formerly APPLY CHANGE INTO)`** instead of a traditional batch job with complex **`MERGE INTO`** `syntax`

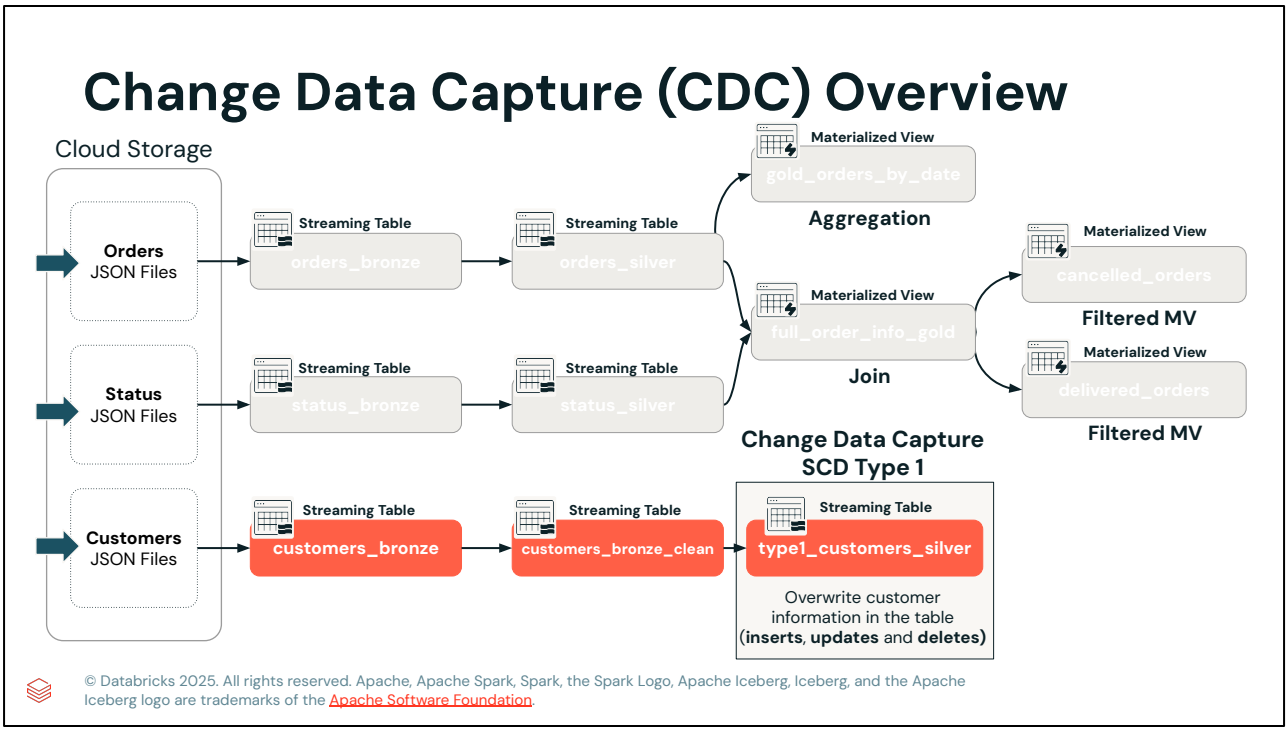
Use AUTO CDC FROM SNAPSHOT (Public Preview as of 2025Q2) to process changes in database snapshots.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

- KEYS (CustomerID) – Defines the primary key used to match rows between source and target.
- AUTO CDC AS DELETE WHEN operation = "DELETE" – Defines when a row should be treated as a delete, based on the operation column.
- SEQUENCE BY ProcessDate – Establishes the event order, critical for applying changes in the correct sequence.
- COLUMNS * EXCEPT (operation) – Includes all columns except the CDC operation indicator in the target.
- STORED AS SCD TYPE 2
  - Stores the history of changes using the SCD Type 2 pattern.
  - If omitted, the default is SCD Type 1 (overwrite only).

Refer to:

- The AUTO CDC APIs: Simplify change data capture with Lakeflow Declarative Pipelines | Databricks on AWS
- AUTO CDC INTO (Lakeflow Declarative Pipelines) | Databricks on AWS



Let's look at the final CDC flow we are adding to our pipeline.

Customers Flow Overview:
1. Ingest customer JSON files - Bring raw data into the pipeline from cloud storage into the **customers_bronze** streaming table.

2. Create **customers_bronze_clean** table - A cleaned streaming table that filters and formats incremental updates, inserts, and deletes from the **customers_bronze** streaming table.

3. Use AUTO CDC INTO **customers_silver**
- Uses SCD Type 1 to overwrite customer changes (updates, inserts, and deletes).
- Implemented using AUTO CDC INTO with STORED AS SCD TYPE 1.

This final flow combines CDC logic with the medallion architecture to maintain an updated customers (with no historical information), critical for downstream analytics, compliance, and personalization.

Refer to: [AUTO CDC INTO (Lakeflow Declarative Pipelines) | Databricks on AWS]

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.